

Formation Complète .NET

Les Bases : C# et SQL

Socle Théorique et Fondations du Langage

Objectifs du Module

- Comprendre l'architecture de la plateforme .NET et le fonctionnement du runtime (CLR).
- Installer et configurer un environnement de développement professionnel (Visual Studio / VS Code).
- Maîtriser la syntaxe fondamentale du langage C# et la modélisation SQL.
- Acquérir une autonomie complète avant d'aborder les projets d'ingénierie logicielle.

Support Pédagogique Révolutionnaire

Édition Académique & Technique — Version .NET 8+

Kevin Özkaraca

Ebéniste spécialisé : C# - SQL - ASP .NET Core - Angular - WPF - MAUI

31 juillet 2026

1 Environnement et Premier Programme

1.1 Installation de l'Environnement de Développement

Pour développer efficacement en C#, le choix et la configuration de l'Environnement de Développement Intégré (IDE) sont cruciaux. Deux options principales s'offrent au développeur :

1.1.1 Visual Studio (Environnement Complet)

Visual Studio est l'IDE de référence pour le développement .NET sous Windows.

1. Télécharger la version **Visual Studio Community** (gratuite pour l'apprentissage et l'usage individuel).
2. Lors de l'installation via le *Visual Studio Installer*, cocher impérativement la charge de travail (*workload*) :

Développement .NET Desktop (comprend le SDK .NET, les outils de débogage et C#).

1.1.2 Visual Studio Code (Éditeur Léger & Multiplateforme)

VS Code est un éditeur de code extensible, idéal pour les environnements Linux, macOS ou Windows légers.

1. Télécharger et installer le **SDK .NET 8+** depuis le site officiel Microsoft.
2. Installer **Visual Studio Code**.
3. Ouvrir l'onglet Extensions (**Ctrl+Shift+X**) et installer la suite d'extensions officielles :
 - **C# Dev Kit** (fournit IntelliSense, la gestion de solutions et le débogage avancé).
 - **C#** (support du langage via OmniSharp/Roslyn).

1.2 Qu'est-ce que C#, .NET, CLI et CLR ?

Pour programmer en C# avec rigueur, il est essentiel d'assimiler l'architecture de la plateforme.

1.2.1 Vue d'Ensemble de l'Écosystème

Tableau : Composants clés de l'écosystème Microsoft .NET

Terme	Nature	Description
C#	Langage	Langage de programmation moderne, orienté objet, à typage fort.
.NET	Plateforme	Écosystème réunissant le runtime, les bibliothèques standard et les outils.
Roslyn	Compilateur	Compilateur officiel traduisant le C# en langage intermédiaire (CIL).
CLI	Norme	<i>Common Language Infrastructure</i> — Norme (ECMA-335) régissant .NET.
CLR	Runtime	<i>Common Language Runtime</i> — Moteur d'exécution virtuel exécutant le code.

1.2.2 Définitions des Concepts Clés

Afin de bien appréhender l'écosystème .NET, il est crucial de comprendre la terminologie employée :

- **Langage typé vs non typé (Typage fort)** : Un **langage typé** (comme C# ou Java) exige que chaque donnée en mémoire possède une nature stricte (entier, texte, booléen) définie à l'avance. À l'inverse, un langage à **typage dynamique** (souvent appelé vulgairement "non typé", comme JavaScript ou Python) permet de changer la nature d'une variable à la volée. C# est un langage **fortement typé** : le compilateur vérifie la cohérence des opérations et interdit les actions illogiques (ex : diviser un texte par un entier) avant même l'exécution, garantissant ainsi une grande fiabilité et robustesse des applications.
- **Runtime (Moteur d'exécution)** : Un *runtime* est un environnement logiciel qui s'exécute en arrière-plan et héberge votre programme. Sans le runtime (le CLR pour .NET), votre code compilé serait incompréhensible pour le système d'exploitation. Le runtime est responsable de la traduction finale des instructions pour le processeur physique, de l'allocation mémoire et de la sécurité logicielle.
- **Bibliothèque (Library)** : Une bibliothèque est une collection de code pré-écrit, optimisé et testé, prêt à être intégré dans vos propres projets pour vous éviter de "réinventer la roue". La plateforme .NET inclut la *Base Class Library* (BCL), offrant des milliers de fonctionnalités prêtes à l'emploi (manipulation de fichiers, connexions réseau, algorithmes cryptographiques).
- **Outils (Tools)** : Ce sont les utilitaires qui accompagnent le développeur tout au long du cycle de production logicielle. Quelques exemples incontournables dans .NET :
 - **NuGet** : Le gestionnaire de paquets officiel, permettant de rechercher, télécharger et intégrer des bibliothèques externes tierces.
 - **CLI dotnet (Command Line Interface)** : L'outil permettant de créer, compiler, tester et déployer des applications .NET depuis un terminal.
 - **MSBuild** : Le moteur de construction sous-jacent qui orchestre la compilation des solutions complexes.
- **Norme (Standard / Spécification)** : Une norme est un document théorique et officiel (tel un cahier des charges). Elle ne contient aucun code exécutable, mais dicte les règles absolues de fonctionnement d'un système. Par exemple, la **CLI** est la norme (le plan de construction architectural), tandis que le **CLR** est l'implémentation de cette norme (le moteur réel construit en respectant rigoureusement ce plan).

1.2.3 La CLI (Common Language Infrastructure) : La Norme Théorique

La **CLI** (*Common Language Infrastructure*) est une spécification internationale qui décrit l'architecture nécessaire pour permettre à plusieurs langages haute performance (C#, F#, VB.NET) de s'exécuter sur la même plateforme sans recompilation spécifique à la machine hôte.

La CLI définit deux piliers fondamentaux :

1. **CIL / IL (Common Intermediate Language)** : Le jeu d'instructions universel auquel tous les langages .NET se réduisent après la première phase de compilation.
2. **CTS (Common Type System)** : La spécification unifiée des types de données (garantissant qu'un entier `int` possède la même représentation mémoire en C# et en F#).

1.2.4 Le CLR (Common Language Runtime) : Le Moteur d'Exécution

Le **CLR** est l'implémentation concrète de la spécification CLI par Microsoft. Il agit comme une machine virtuelle hautement optimisée qui prend en charge :

- **La Compilation JIT (Just-In-Time)** : Traduction à la volée du code IL en instructions machines natives (0 et 1) adaptées au processeur cible (x64, ARM64).
- **Le Garbage Collector (GC)** : Gestion automatique de la mémoire et libération des objets inutilisés.
- **La Sécurité de Typage** : Vérification que le code n'accède pas à des zones mémoire non autorisées.

1.2.5 Les 4 Piliers de l'Architecture .NET

Voici le rôle exact de chaque composant dans le cycle de vie de votre code :

- **La CLI (L'Infrastructure)** : C'est le cahier des charges. Elle dicte les règles que tout langage .NET doit respecter (les types de données, la structure du code). Elle garantit que le code C# pourra cohabiter avec du code F#.
- **Roslyn (Le Compilateur)** : C'est l'outil qui lit votre code source C# (.cs). Il vérifie qu'il n'y a pas d'erreurs de syntaxe, puis le traduit en **Code IL** (un code intermédiaire compréhensible par la machine virtuelle).
- **Le CLR (Le Moteur d'Exécution)** : C'est le cœur de .NET. Il prend le Code IL généré par Roslyn et le fait tourner. Il s'occupe de gérer la mémoire (Garbage Collector) et d'assurer la sécurité du programme pendant qu'il tourne.
- **Le JIT (Le Traducteur Final)** : C'est un composant interne au CLR. Juste avant l'exécution, il traduit le Code IL en instructions binaires ultra-rapides, spécifiquement optimisées pour le processeur de la machine sur laquelle le programme tourne (Intel, AMD, ARM).

1.3 Structure d'un Programme C#

Tout programme C# s'articule autour de trois éléments fondamentaux : l'espace de noms (namespace), la classe (class), et le point d'entrée (Main).

1.3.1 Structure Classique (Explicite)

Voici le squelette standard d'une application C# console :

```

1 using System;
2
3 namespace LesBases.Chapitre1
4 {
5     internal class Program
6     {
7         static void Main(string[] args)
8         {
9             // Point d'entree principal de l'application
10            Console.WriteLine("Bienvenue dans la formation C# !");
11        }
12    }
13 }
```

Listing 1 – Structure classique d'un fichier C# (Program.cs)

Explication détaillée du code :

- `using System;` : En C#, le code est organisé en bibliothèques (espaces de noms). Cette ligne dit au compilateur : *"Je veux utiliser les outils fournis par Microsoft, notamment ceux pour écrire dans la console"*. Sans cette ligne, il faudrait écrire `System.Console.WriteLine(...)`.

- `namespace LesBases.Chapitre1` : Un **namespace** (espace de noms) est un dossier virtuel. Il permet de ranger vos classes pour que votre code ne se mélange pas avec celui d'autres développeurs. Si deux classes s'appellent `Program`, le namespace permet de les différencier.
- `internal class Program` : En C# classique, tout doit obligatoirement être contenu dans une "classe". La classe `Program` est traditionnellement le point de départ. Le mot-clé `internal` signifie que cette classe n'est visible que dans ce projet.
- `static void Main(string[] args)` : C'est le point d'entrée critique de l'application. Quand vous lancez le programme, le moteur de .NET cherche très exactement cette méthode `Main` et exécute le code qui se trouve à l'intérieur de ses accolades.

1.3.2 Structure Moderne : Top-Level Statements (C# 9+ / .NET 8)

Dans les versions modernes de C#, le boilerplate (code répétitif) peut être omis pour les scripts simples et les points d'entrée :

```
1 // Les instructions sont directement au niveau racine
2 Console.WriteLine("Bienvenue dans la formation C# moderne !");
```

Listing 2 – Structure moderne avec Top-Level Statements

Quelle structure choisir et comment l'afficher ?

Aujourd'hui, **Microsoft recommande d'utiliser les Top-Level Statements** (la structure moderne) pour les nouveaux projets. C'est plus clair et cela enlève le code inutile pour démarrer. C'est le comportement par défaut quand on crée un nouveau projet .NET.

Cependant, Roslyn cache simplement l'ancienne structure : il génère toujours une classe `Program` et une méthode `Main` en arrière-plan pendant la compilation.

Comment revenir à la structure classique ? Si votre projet nécessite une configuration complexe au démarrage ou si vous préférez l'ancienne méthode, vous pouvez forcer la structure classique :

- Dans Visual Studio, lors de la création du projet, cochez : *"Do not use top-level statements"*.
- En ligne de commande (CLI), tapez : `dotnet new console -use-program-main`.

1.4 Le Cycle de Compilation et d'Exécution

Le processus complet de transformation du code source C# en exécution système suit un pipeline strict en deux phases :

1. Phase 1 : Compilation Statique (Roslyn)

- Analyse syntaxique et sémantique du fichier `.cs`.
- Génération de l'assemblage (fichier `.dll` ou `.exe`).
- L'assemblage contient le **Code IL** et les **Métadonnées**.

2. Phase 2 : Exécution Dynamique (CLR / JIT)

- Chargement de l'assemblage par le CLR.
- Le compilateur ****JIT**** traduit les blocs IL nécessaires en instructions machines natives.
- Le processeur exécute le code natif.

1.5 Synthèse du Chapitre

Tableau : Récapitulatif des notions clés du Chapitre 1

Concept	A retenir
Roslyn	Compilateur C# qui transforme le code .cs en IL .
Code IL	Langage intermédiaire universel de la plateforme .NET.
CLR	Moteur d'exécution qui gère la mémoire (GC) et compile le code IL via le JIT .
Main()	Point d'entrée obligatoire de toute application console C#.

2 Variables et Types de Données

Les variables constituent le fondement de la programmation impérative. Elles permettent d'allouer de l'espace mémoire pour stocker, lire et manipuler des données tout au long de l'exécution d'un programme. En C#, langage à typage statique fort, chaque variable doit être déclarée avec un type précis, garantissant la sûreté du code dès la compilation.

2.1 Déclaration et Initialisation

La déclaration d'une variable exige de spécifier son **type** suivi de son **identifiant** (nom). L'initialisation consiste à lui attribuer une valeur en mémoire via l'opérateur d'affectation `=`.

```

1 // 1. Déclaration (allocation mémoire statique)
2 int age;
3 string identifiantSysteme;
4
5 // 2. Initialisation (affectation de valeur)
6 age = 30;
7 identifiantSysteme = "SYS_4099";
8
9 // 3. Déclaration et initialisation combinées (Recommandé)
10 double latenceReseau = 19.99;
```

Listing 3 – Déclaration et initialisation standard

2.2 Les Types de Données Primitifs

Le système unifié des types (CTS - Common Type System) de .NET propose de multiples types adaptés aux différents besoins de précision, d'espace mémoire et de logique métier.

2.2.1 Types Fondamentaux

Tableau : Les types de données les plus courants

Type	Nature	Exemple d'utilisation
int	Entier signé 32-bit	int compteurRequetes = 42;
string	Chaîne de caractères (UTF-16)	string nomClient = "Alice";
bool	Booléen (logique)	bool accesAutorise = true;
double	Décimal (64-bit IEEE 754)	double ratio = 3.14159;

2.2.2 Types Spécifiques et Précision

Pour des besoins d'ingénierie particuliers, C# propose des types avec des capacités mémoire étendues ou des gestions d'arrondis spécifiques. Le suffixe de type (**m**, **f**, **L**) est obligatoire lors de l'affectation littérale.

Tableau : Types avancés et suffixes

Type	Spécificité	Exemple
decimal	Haute précision	decimal prix = 19.99m;
float	Simple précision (32-bit)	float delta = 9.81f;
long	Grand entier (64-bit)	long idBaseDonnees = 9876543210L;
char	Caractère unique	char separateur = ',';

Critères de choix : float, double et decimal

- **float** : Utilisé principalement pour la 3D, le rendu graphique ou le traitement du signal (la vitesse de calcul CPU/GPU prime sur l'exactitude extrême).
- **double** : Type décimal par défaut en C#, idéal pour les calculs mathématiques et la trigonométrie.
- **decimal** : Implémente une arithmétique en base 10. Strictement indispensable pour les calculs monétaires et financiers afin d'éviter les pertes de précision liées au format flottant binaire.

2.3 Inférence de Type avec var

L'instruction **var** demande au compilateur de déduire automatiquement le type de la variable en analysant l'expression à droite de l'opérateur d'affectation. Cela allège l'écriture sans sacrifier la sécurité.

```
1 // Le compilateur (Roslyn) déduit 'int'
2 var compteur = 0;
3
4 // Le compilateur déduit 'string'
5 var identifiantSession = "USER_902";
```

Listing 4 – Typage statique implicite avec var

Avertissement Architectural : var n'est pas dynamique

Le mot-clé **var** maintient un typage **statique fort**. Lors de la compilation, l'instruction **var x = 10;** est transformée en **int x = 10;**. Il sera donc impossible d'affecter une chaîne de caractères à **x** ultérieurement.

2.4 Les Constantes (const)

Une constante alloue un espace mémoire immuable. Sa valeur est résolue et scellée lors de la compilation. Elle ne pourra jamais être modifiée durant l'exécution, garantissant ainsi l'intégrité de la donnée.

```
1 const double Pi = 3.14159;
2 const int MaxRequetesParSeconde = 500;
```

Listing 5 – Déclaration de constantes

2.5 Manipulation des Chaînes de Caractères

En C#, les chaînes (**string**) peuvent être combinées (concaténation) à l'aide de l'opérateur **+**. Notez que l'API de base permet également de lire les entrées de l'utilisateur.

```
1 string prenom = "Alan";
2 string nom = "Turing";
3
4 // Concaténation standard
5 string nomComplet = prenom + " " + nom;
6 Console.WriteLine("Utilisateur : " + nomComplet);
7
8 // Récupération d'une saisie dans le flux Standard Input (stdin)
9 Console.WriteLine("Entrez votre code d'accès :");
```



```

10 string? codeSaisi = Console.ReadLine(); // 'string?' autorise
    explicitement la valeur null

```

Listing 6 – Concaténation et récupération d'entrées

2.6 La Portée des Variables (Scope)

La portée d'une variable définit son accessibilité et sa durée de vie en mémoire. C# définit des périmètres d'accès très stricts, indispensables à l'encapsulation.

Tableau : Les niveaux de portée en C#

Type de portée	Zone de déclaration	Durée de vie
Locale	Dans une méthode ou un bloc { }	Détruite à la fin du bloc d'exécution.
Champ d'instance	Dans la classe (hors méthode)	Détruite avec l'instance de l'objet (via le GC).
Statique	Dans la classe (avec <code>static</code>)	Réside en mémoire jusqu'à la fermeture du processus.

2.6.1 Démonstration des Portées et du Masquage (Shadowing)

```

1 using System;
2
3 class ServeurWeb
4 {
5     // 1. Champ statique : Partagé par toutes les instances de
    ServeurWeb en mémoire
6     private static int _requetesGlobales = 0;
7
8     // 2. Champ d'instance : Propre à une instance spécifique du serveur
9     private string _adresseIp;
10
11     public ServeurWeb(string adresseIp)
12     {
13         _adresseIp = adresseIp;
14     }
15
16     public void TraiterRequete(string _adresseIp)
17     {
18         // 3. Masquage (Shadowing) : Le paramètre masque le champ d'
    instance
19
20         // Incrémentation de la variable statique partagée
21         _requetesGlobales++;
22
23         // 4. Variable locale : Isolée dans le bloc de cette méthode
24         int taillePayload = 1024;
25
26         // Pour accéder au champ d'instance masqué, l'usage de 'this'
    est obligatoire
27         Console.WriteLine($"Connexion de {_adresseIp} sur l'hôte {this.
    _adresseIp}");
28     }
29 }

```

Listing 7 – Gestion de la portée, variables statiques et masquage

2.7 Le Type dynamic

Contrairement à `var`, le mot-clé `dynamic` désactive intégralement la vérification statique du compilateur. Le type n'est résolu qu'au moment de l'exécution (Runtime).

```
1 dynamic variableDynamique = 10;  
2 variableDynamique = "Transformation en chaîne de caractères"; // Aucune  
   erreur de compilation
```

Listing 8 – Désactivation du typage avec `dynamic`

L'usage de `dynamic` doit être rigoureusement limité à des cas d'ingénierie spécifiques (interopérabilité COM, réflexion, traitement de structures JSON non typées) car il annule toute la sécurité offerte par le langage.

2.8 Conversion de Types (Casting)

La conversion de types est une opération courante, notamment lors de l'intégration de flux de données externes.

```
1 // 1. Conversion Implicite (Sûre - Garantie sans perte d'information)  
2 int entier = 10;  
3 double reel = entier; // L'entier (32-bit) s'intègre sans perte dans le  
   décimal (64-bit)  
4  
5 // 2. Conversion Explicite ou Cast (Avertissement : risque de troncature  
   )  
6 double valeurMesuree = 9.7;  
7 int valeurTronquee = (int)valeurMesuree; // Vaut 9 (la fraction décimale  
   est détruite)  
8  
9 // 3. Conversion par Analyse (Parsing : Chaîne vers Numérique)  
10 string donneesReseau = "1024";  
11 int portServeur = int.Parse(donneesReseau);
```

Listing 9 – Mécanismes de conversion en C#

2.9 Bonnes Pratiques de Codage

Règles strictes de développement C#

- **Variables non initialisées** : Le compilateur C# refuse catégoriquement l'utilisation d'une variable locale non assignée (**Use of unassigned local variable**). Les champs de classe, eux, prennent par défaut la valeur 0 ou `null`.
- **Confusion Affectation / Comparaison** : Ne jamais confondre l'opérateur d'affectation `=` avec l'opérateur d'égalité logique `==`.
- **Déclarations en ligne** : L'écriture `int a = 1, b = 2;` est permise par le compilateur mais prohibée par les conventions de Clean Code. L'architecture logicielle exige une déclaration par ligne pour préserver la lisibilité de l'historique de versionnage (Git).

3 Conventions de Nommage, Formatage et Clean Code

Écrire du code informatique ne se limite pas à produire une séquence d'instructions exécutables par le compilateur. Dans l'ingénierie logicielle professionnelle, le code C# doit être **lisible**, **maintenable** et **homogène**. Les conventions de style constituent le langage commun facilitant la collaboration au sein des équipes de développement.

3.1 Conventions de Nommage Officielles

Le nommage d'un élément logiciel doit rendre le code auto-documenté. Microsoft définit des règles précises de casse selon la nature et la portée de chaque symbole.

3.1.1 Les Styles de Casse en C#

Tableau : Règles de casse officielles C# (Microsoft Design Guidelines)

Style	Format	Exemple	Cas d'utilisation
PascalCase	Majuscule à chaque mot	CalculerPrixTotal	Classes, structures, enums, méthodes, propriétés, événements.
camelCase	1 ^{er} mot en minuscule	prixUnitaire	Variables locales, paramètres de méthodes.
_camelCase	camelCase précédé de _	_compteurClient	Champs privés d'une classe (<i>private fields</i>).
IPascalCase	PascalCase avec préfixe I	IDisposable	Interfaces.
PascalCase	Majuscule à chaque mot	NombreMaxEssais	Constantes (const et readonly).

3.1.2 Exemples de Conformité Syntaxique

```
1 // Interface : Prefix 'I' + PascalCase
2 public interface ICalculable
3 {
4     // Propriete : PascalCase
5     decimal Total { get; }
6 }
7
8 // Classe : PascalCase
9 public class GestionnaireCommande : ICalculable
10 {
11     // Constante : PascalCase
12     public const int LimiteArticles = 100;
13
14     // Champ prive : _camelCase
15     private readonly string _identifiantCommande;
16     private int _nombreArticles;
17
18     // Propriete publique : PascalCase
19     public decimal Total { get; private set; }
20
21     // Constructeur : parametre en camelCase
22     public GestionnaireCommande(string identifiantCommande)
23     {
24         _identifiantCommande = identifiantCommande;
25     }
26
27     // Methode : PascalCase, parametre : camelCase, variable locale :
28     // camelCase
29     public void AjouterArticle(decimal prixUnitaire, int quantite)
30     {
31         decimal sousTotal = prixUnitaire * quantite;
32         Total += sousTotal;
33         _nombreArticles += quantite;
34     }
35 }
```

Listing 10 – Conventions de nommage recommandées en C#

Pièges de nommage à éviter

- **Ne pas utiliser le snake_case** (`prix_total`) : réservé à d'autres langages (Python, Rust).
- **Éviter la notation hongroise** (`strNom`, `intAge`) : le typage C# est déjà statique et explicite.
- **Omettre le préfixe I sur les interfaces** : rend la distinction impossible avec les classes abstraites.

3.2 Conventions de Formatage (Style Allman)

Le formatage régit l'organisation visuelle du code source. C# adopte historiquement le style d'accolades **Allman**.

3.2.1 Règles Fondamentales d'Organigramme Visuel

1. **Accolades Allman** : L'accolade ouvrante { se place **toujours sur sa propre ligne**, alignée verticalement avec la structure parente.
2. **Indentation** : Strictement 4 espaces par niveau de bloc (ne pas mélanger tabulations et espaces).
3. **Espacement des opérateurs** : Un espace unique de part et d'autre des opérateurs binaires (`=`, `+`, `==`, `&&`).

```

1 // [CORRECT] Style Allman officiel C#
2 public void IncrireUtilisateur(string nom)
3 {
4     if (string.IsNullOrEmpty(nom))
5     {
6         throw new ArgumentException("Nom invalide", nameof(nom));
7     }
8 }
9
10 // [INCORRECT] Style K&R (Anti-pattern en C#)
11 public void IncrireUtilisateur(string nom) {
12     if (string.IsNullOrEmpty(nom)) {
13         throw new ArgumentException("Nom invalide", nameof(nom));
14     }
15 }
```

Listing 11 – Comparaison de formatage : Allman (C#) vs K&R (Java/JS)

3.3 Organisation Interne d'une Classe C#

Une classe lisible regroupe ses membres selon un ordre standardisé reconnu par la communauté.

Tableau : Ordre d'agencement recommandé des membres d'une classe

Ordre	Catégorie de Membre	Description
1	Constantes	public const et private const.
2	Champs privés	Champs d'instance d'encapsulation (<code>_camelCase</code>).
3	Constructeurs	Initialisateurs d'instance.
4	Propriétés publiques	Accesseurs de données (<code>get</code> , <code>set</code> , <code>init</code>).
5	Méthodes publiques	Points d'entrée de la logique métier.
6	Méthodes privées	Méthodes utilitaires internes d'assistance.

```

1 namespace LesBases.Services
2 {
3     public class ServiceCompteBancaire
4     {
5         // 1. Constantes
6         private const decimal SoldeMinimumAutorise = -500.00m;
7
8         // 2. Champs privés
9         private readonly string _numeroCompte;
10        private decimal _solde;
11
12        // 3. Constructeurs
13        public ServiceCompteBancaire(string numeroCompte, decimal
soldeInitial)
14        {
15            _numeroCompte = numeroCompte;
16            _solde = soldeInitial;
17        }
18
19        // 4. Proprietes publiques
20        public string NumeroCompte => _numeroCompte;
21        public decimal Solde => _solde;
22
23        // 5. Methodes publiques
24        public void Deposier(decimal montant)
25        {
26            ValiderMontantPositif(montant);
27            _solde += montant;
28        }
29
30        // 6. Methodes privees d'assistance
31        private static void ValiderMontantPositif(decimal montant)
32        {
33            if (montant <= 0)
34            {
35                throw new ArgumentOutOfRangeException(nameof(montant), "
Le montant doit etre positif.");
36            }
37        }
38    }
39 }

```

Listing 12 – Exemple de structure canonique d'un service C#

3.4 Inférence de Type (`var`) vs Type Explicite

Le mot-clé `var` (introduit en C# 3) permet l'inférence de type à la compilation. Son utilisation est encadrée par des critères d'évidence visuelle.

Règle d'or de l'inférence var

Utiliser `var` uniquement lorsque le type retourné est **immédiatement explicite** à la lecture du côté droit de l'affectation (opérateur `new` ou littéral). Dans le cas contraire, conserver le type explicite pour maintenir la clarté.

```
1 // [BON] Le type est evident via l'instanciation directe ou le littéral
2 var nomUtilisateur = "Alice"; // string
3 var listeClients = new List<string>(); // List<string>
4 var dictionnaire = new Dictionary<int, string>(); // Evite la
    repetition inutile
5
6 // [MAUVAIS] Le type est masque par l'appel de methode
7 var resultat = CalculerIndiceForme(); // De quel
    type est resultat ? int ? double ? CustomType ?
8
9 // [CORRECT] Type explicite quand l'intention doit être clarifiée
10 decimal indiceForme = CalculerIndiceForme();
```

Listing 13 – Usage correct et incorrect de var

3.5 Contrôle de Flux et Retours Anticipés (*Early Return*)

Pour éviter l'empilement excessif de conditions imbriquées (syndrome de la pyramide d'indentation), C# préconise la technique des **retours anticipés**.

```
1 // [MAUVAIS] Imbrication profonde et illisible
2 public string TraiterCommande(Commande cmd)
3 {
4     if (cmd != null)
5     {
6         if (cmd.EstValidee)
7         {
8             if (cmd.StockDisponible)
9             {
10                 return "Commande expediee";
11             }
12             else
13             {
14                 return "Rupture de stock";
15             }
16         }
17         else
18         {
19             return "Commande non validee";
20         }
21     }
22     return "Erreur commande nulle";
23 }
24
25 // [EXCELLENT] Structure avec retours anticipés (Early Return)
26 public string TraiterCommandePropre(Commande cmd)
27 {
28     if (cmd == null) return "Erreur commande nulle";
29     if (!cmd.EstValidee) return "Commande non validee";
30     if (!cmd.StockDisponible) return "Rupture de stock";
31
32     return "Commande expediee";
33 }
```

33 }

Listing 14 – Réfactorisation d’une méthode complexe par Early Return

3.6 Gestion Rigoureuse des Exceptions

La gestion des erreurs doit respecter la chaîne de rétropropagation des exceptions sans altérer la pile d’appels (*stack trace*).

Règle d’or : `throw;` vs `throw ex;`

Lors du re-lancement d’une exception dans un bloc `catch`, utiliser strictement `throw;` sans argument. L’instruction `throw ex;` réinitialise la pile d’appels au niveau du `catch`, masquant l’origine exacte du plantage.

```
1 try
2 {
3     ExecuterTraitementReseau();
4 }
5 catch (TimeoutException ex)
6 {
7     // [BON] Conserve l'intégralité de la stack trace originale
8     Logger.LogError(ex, "Timeout reseau detecte");
9     throw;
10 }
11 catch (Exception ex)
12 {
13     // [INCORRECT] Ecraser la stack trace originale
14     // throw ex; <-- NE JAMAIS FAIRE CELA
15 }
```

Listing 15 – Bonnes pratiques de re-lancement d’exceptions

3.7 Outillage Automatisé (.editorconfig & Roslyn)

Afin de garantir le respect des conventions sans effort manuel, l’écosystème .NET utilise des fichiers de configuration standardisés à la racine du dépôt.

3.7.1 Fichier de Configuration .editorconfig

```
1 root = true
2
3 [*.*]
4 # Indentation
5 indent_style = space
6 indent_size = 4
7
8 # Regles de nommage des champs privés
9 dotnet_naming_rule.private_fields_should_have_underscore.severity =
    warning
10 dotnet_naming_rule.private_fields_should_have_underscore.symbols =
    private_fields
11 dotnet_naming_rule.private_fields_should_have_underscore.style =
    prefix_underscore
12
13 dotnet_naming_symbols.private_fields.applicable_kinds = field
```

```

14 dotnet_naming_symbols.private_fields.applicable_accessibilities =
    private
15
16 dotnet_naming_style.prefix_underscore.required_prefix = _
17 dotnet_naming_style.prefix_underscore.capitalization = camel_case
18
19 # Respect obligatoire des accolades Allman
20 csharp_prefer_braces = true:warning

```

Listing 16 – Exemple de fichier .editorconfig standard pour projet C#

3.7.2 Commande de Formatage CLI

La CLI .NET intègre un outil natif pour corriger le formatage du projet entier :

```

1 # Reformate automatiquement tous les fichiers de la solution selon le .
  editorconfig
2 dotnet format
3
4 # Verification sans modification (ideal pour les pipelines CI/CD)
5 dotnet format --verify-no-changes

```

Listing 17 – Commandes dotnet format

3.8 Synthèse des Conventions

Tableau : Tableau récapitulatif des règles d'ingénierie C#

Élément	Convention Officielle	Exemple Validé
Classe / Structure	PascalCase	class CompteUtilisateur
Interface	I + PascalCase	interface IRepository
Méthode / Propriété	PascalCase	public void Sauvegarder()
Variable locale	camelCase	int totalElements = 0;
Champ privé	_camelCase	private string _cleApi;
Accolades	Style Allman	Première ligne dédiée
Re-throw exception	throw;	Préserve la pile d'appels

4 Passage de Paramètres : ref et out

En C#, il est essentiel de comprendre comment les données voyagent entre les différentes méthodes de votre programme. Par défaut, lorsque vous transmettez une variable de type valeur (comme un `int`, un `bool` ou un `double`) en tant qu'argument à une méthode, le compilateur effectue un **passage par valeur** (souvent appelé passage par copie).

Concrètement, cela signifie que le moteur d'exécution clone la valeur de votre variable d'origine et ne transmet que ce clone à la méthode appelée. Cette méthode va alors stocker cette copie de travail isolée dans sa propre zone de mémoire temporaire, appelée la pile d'exécution (*Stack*).

Par conséquent, si la méthode altère cette donnée au cours de son exécution (en l'incrémentant ou en l'écrasant), elle ne modifie que **sa propre copie locale**. La variable d'origine, appartenant à la méthode appelante, demeure totalement intacte et protégée en mémoire.

Cependant, dans certaines architectures logicielles, ce comportement par défaut s'avère limitant. Un développeur peut avoir le besoin légitime qu'une méthode :

- **Modifie directement** l'état de la variable d'origine qui lui a été confiée.
- **Exporte plusieurs résultats distincts** simultanément, outrepassant ainsi la limite naturelle de l'instruction unique `return`.

C'est pour répondre à ces deux besoins spécifiques de manipulation de la mémoire que le langage C# met à disposition les modificateurs de paramètres `ref` et `out`.

4.1 Le modificateur ref (Passage par référence)

4.1.1 Introduction Théorique

Le mot-clé `ref` ordonne au compilateur de transmettre non pas la valeur de la variable, mais son **adresse mémoire** (un pointeur managé). La méthode accède ainsi directement à l'espace mémoire alloué par l'appelant. Une contrainte stricte s'applique : la variable cible doit obligatoirement être initialisée par l'appelant *avant* d'être passée en paramètre, car la méthode appelée peut choisir de lire sa valeur initiale avant de la modifier.

4.1.2 Syntaxe et Règles

Le mot-clé `ref` doit être déclaré explicitement dans la signature de la méthode, ainsi qu'au moment de l'appel. Cette verbosité intentionnelle signale clairement au développeur lisant le code que la variable est susceptible de subir des mutations (effets de bord).

```

1 // 1. Declaration de la methode avec le modificateur 'ref'
2 void IncrementerCompteur(ref int compteur)
3 {
4     compteur += 10; // Modification directe de l'adresse memoire cible
5 }
6
7 // 2. Initialisation prealable obligatoire
8 int tentatives = 5;
9
10 // 3. Appel avec le modificateur 'ref' explicite
11 IncrementerCompteur(ref tentatives);
12
13 Console.WriteLine(tentatives); // Affiche : 15

```

Listing 18 – Utilisation basique de ref

4.1.3 Exemple d'Ingénierie : Algorithme de Permutation (Swap)

Le passage par référence est l'unique solution permettant d'écrire un algorithme d'échange de variables (*Swap*) opérant directement sur l'état externe.

```
1 void EchangerAdresses(ref string adresseA, ref string adresseB)
2 {
3     // Utilisation d'un tampon local pour la permutation
4     string tampon = adresseA;
5     adresseA = adresseB;
6     adresseB = tampon;
7 }
8
9 string serveurPrimaire = "192.168.1.10";
10 string serveurSecondaire = "10.0.0.5";
11
12 EchangerAdresses(ref serveurPrimaire, ref serveurSecondaire);
13
14 Console.WriteLine($"Primaire : {serveurPrimaire} | Secondaire : {
15     serveurSecondaire}");
16 // Le routage est inverse en memoire de maniere permanente
```

Listing 19 – Algorithme d'échange via passage par référence

Pièges courants : Oubli d'initialisation

Le compilateur Roslyn émettra l'erreur stricte CS0165 : Use of unassigned local variable si vous tentez de passer une variable non initialisée avec `ref`. Le CLR exige une adresse mémoire contenant une donnée typée valide.

4.2 Le modificateur `out` (Paramètre de sortie)

4.2.1 Introduction Théorique

Le modificateur `out` permet également le passage par référence, mais impose un contrat sémantique inversé : son but exclusif est **d'exporter des données** hors de la méthode. Contrairement à `ref`, la variable passée via `out` n'a pas besoin d'être initialisée par l'appelant. En contrepartie, le compilateur force contractuellement la méthode appelée à y assigner une valeur avant toute instruction de retour (`return` ou fin de bloc d'exécution).

4.2.2 Syntaxe et Cas d'Usage : Le Pattern Try-Parse

L'usage le plus incontournable du modificateur `out` réside dans les méthodes de conversion sécurisées, nommées *TryParse*. Ces méthodes utilisent leur type de retour classique (`bool`) pour signaler le succès ou l'échec de l'opération, tout en injectant le résultat du calcul dans la variable `out`.

```
1 string chargeUtileReseau = "1024";
2
3 // C# 7+ : Declaration "inline" de la variable 'out' directement lors de
4 // l'appel
5 bool conversionReussie = int.TryParse(chargeUtileReseau, out int
6     portServeur);
7
8 if (conversionReussie)
9 {
10     Console.WriteLine($"Connexion etablie sur le port : {portServeur}");
11 }
```

```

10 else
11 {
12     Console.WriteLine("Erreur de parsing : La donnée réseau est
        corrompue.");
13 }

```

Listing 20 – Pattern Try-Parse avec déclaration inline

Analyse de l'implémentation :

- L'appel à `int.TryParse` tente de parser la chaîne de caractères brute.
- Si le contenu échoue à la conversion (ex : caractères non numériques), la méthode retourne **false** et la variable `portServeur` reçoit la valeur par défaut du type (0). Aucune exception n'est levée, économisant les ressources CPU.
- Si le parsing réussit, la valeur convertie est affectée à l'adresse de `portServeur` et la méthode retourne **true**.

Pièges courants : Rupture du contrat out

L'erreur de compilation CS0177 : `The out parameter must be assigned to before control leaves the current method` survient inévitablement si le code de la méthode appelée omet d'affecter une valeur au paramètre **out**. Le compilateur garantit ainsi la sécurité mémoire et empêche de récupérer une valeur non allouée.

4.3 Évolution Architecturale : Tuples et Bonnes Pratiques

Bien que les paramètres **out** aient été historiquement employés pour retourner de multiples valeurs, l'architecture C# moderne (depuis C# 7) préconise désormais l'utilisation des **Tuples** pour des raisons de syntaxe fonctionnelle et de lisibilité.

Par conséquent, l'usage des paramètres **out** doit être strictement restreint au pattern canonique **TryParse** (ou scénarios de validation similaires). De son côté, l'emploi de **ref** doit être réservé à l'optimisation des performances de bas niveau (ex : éviter de copier des structures très lourdes en mémoire, ou manipuler des buffers d'entrées/sorties natives).

4.4 Synthèse du Chapitre

Tableau : Comparaison des modificateurs de paramètres

Modificateur	Initialisation par l'appelant	Contrat de la méthode
Aucun (par valeur)	Obligatoire	Travaille sur une copie isolée.
ref	Obligatoire	Modifie directement l'adresse mémoire source.
out	Optionnelle (déclaration <i>inline</i> permise)	Obligation absolue d'assigner une valeur avant de retourner.